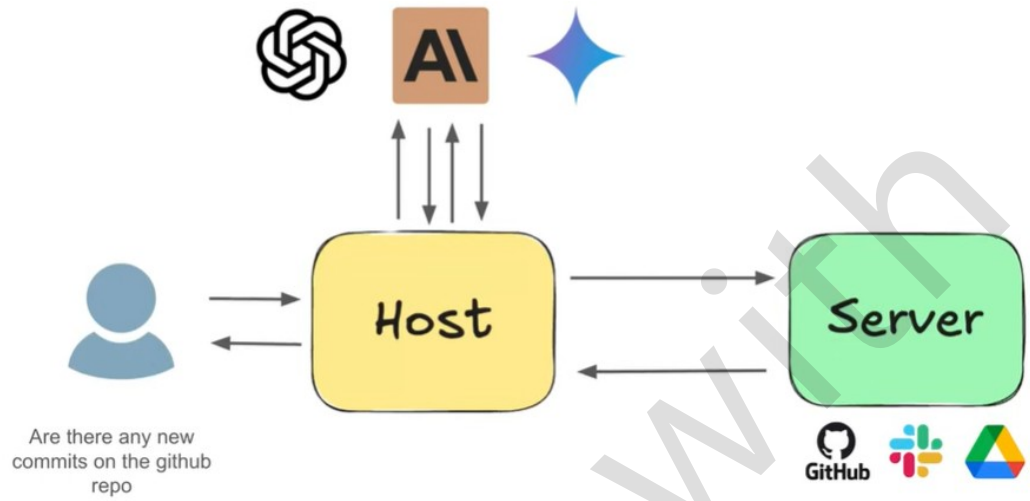
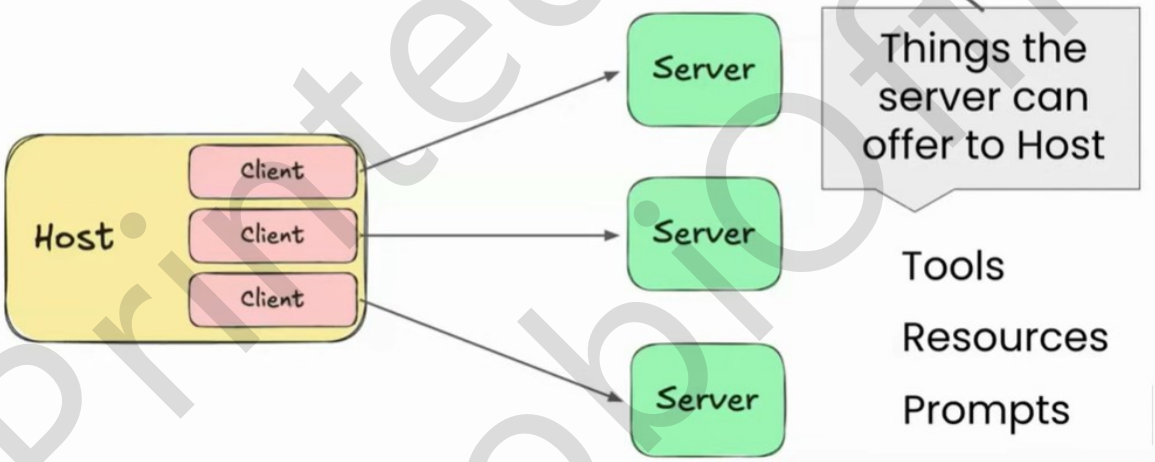
Architecture

Simplest Version

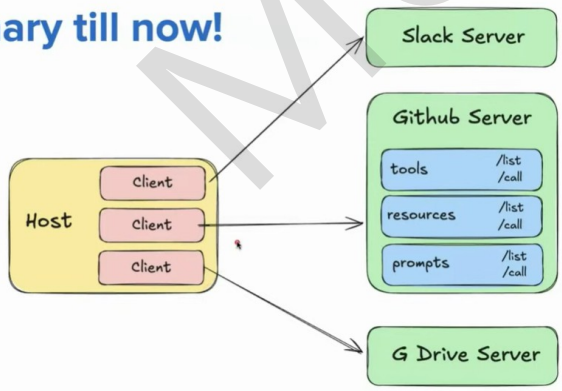


Architecture

Primitives



Summary till now!



MCP Primitives

Tools - Actions the AI ask the server to perform.

Resources - Structured data sources that the AI can read

Prompts - Predefined prompt templates or instructions that the server offers to help shape the AI's behavior

Primitives - Standard Operations

- Tools
  - `tools/list` → Client asks the Server: *"What tools do you provide?"*
  - `tools/call` → Client tells the Server: *"Please run this tool with these arguments."*
- Resources
  - `resources/list` → Client asks: *"What resources are available?"*
  - `resources/read` → Client says: *"Give me the content of this resource."*
  - `resources/subscribe` / `unsubscribe` → Client subscribes or unsubscribes from updates.
- Prompts
  - `prompts/list` → Client asks: *"What prompt templates do you provide?"*
  - `prompts/get` → Client fetches a specific prompt template.

The Prompts Primitives

```
{
  "name": "issue_report_prompt",

  "description": "Write clear, detailed GitHub issues",

  "messages": [{
        "role": "system",

        "content": "Always include: Title, Steps to
Reproduce, Expected, Actual, Environment"}
  }]
}
```

```
Title: Bug – Login Button Not Working

**Steps to Reproduce:**
1. Open the login page
2. Enter valid credentials
3. Click the Login button

**Expected Behavior:**
User should be logged in and redirected to dashboard.

**Actual Behavior:**
Nothing happens when clicking the Login button.

**Environment:**
Chrome 121, macOS 14.2
```

MCP Data Layer

Data Layer - The data layer is the language and grammar of the MCP ecosystem that everyone agrees upon to communicate.

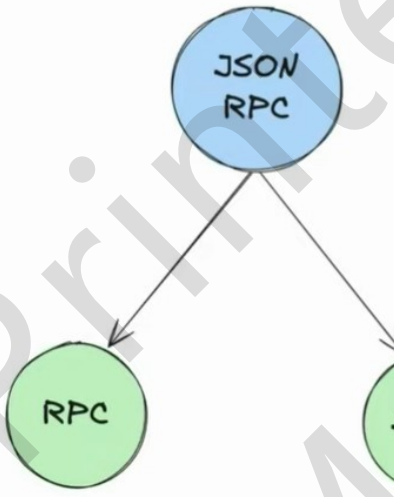
In MCP, **JSON RPC 2.0** serves as the foundation of the data layer

JSON RPC 2.0

JSON-RPC stands for **JavaScript Object Notation – Remote Procedure Call**.

A **Remote Procedure Call (RPC)** allows a program to execute a function on another computer as if it were local, hiding the details of network communication and data transfer. This abstraction makes it easier to build distributed applications.

JSON RPC 2.0



JSON-RPC combines the concept of **Remote Procedure Calls** with the simplicity of **JSON**, allowing developers to structure RPC requests and responses in a standardized JSON format.

JSON RPC 2.0

Request Structure

```
{
  "jsonrpc": "2.0",
  "method": "add",
  "params": [2, 3],
  "id": 1
}
```

Response Structure

```
{
  "jsonrpc": "2.0",
  "result": 5,
  "id": 1
}
```

```
{
  "jsonrpc": "2.0",
  "error": { "code": -32601, "message": "Method not found" },
  "id": 1
}
```

Why JSON RPC for Data Layer?

It's lightweight

Supports bi-directional communication

It is transport-agnostic

Supports batching

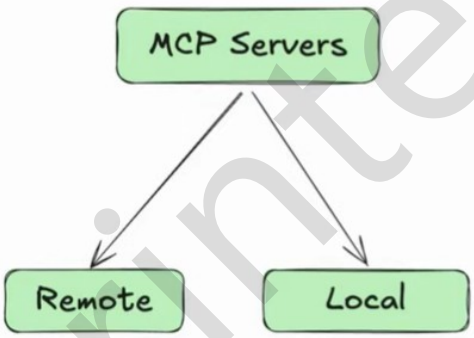
Supports notification

MCP Transport Layer

The **Transport Layer** is the mechanism that moves JSON-RPC messages between the Client and Server.

The choice of transport depends on the type of server.

MCP - Types of Server



A **local server** is a program running on your own computer.

A **remote server** is a program running on another computer (somewhere else on the network or internet) that you connect to over a network.

Local Servers - STDIO

STDIO refers to the built-in streams every program has.

stdin (input the program reads)

stdout (output the program writes)

In MCP, these streams are used as transport layer between the client and server.

How does STDIO work?

The host launches the server as a subprocess on the same machine.

The host(client) writes JSON-RPC messages into the server's STDIN

The server reads those messages, processes them, and writes back responses to it's STDOUT

Remote Servers - HTTP + SSE

Using HTTP allows the host to reach Servers running anywhere

Host sends JSON RPC requests using POST requests with a JSON payload

The transport supports standard HTTP auth methods(like API keys)

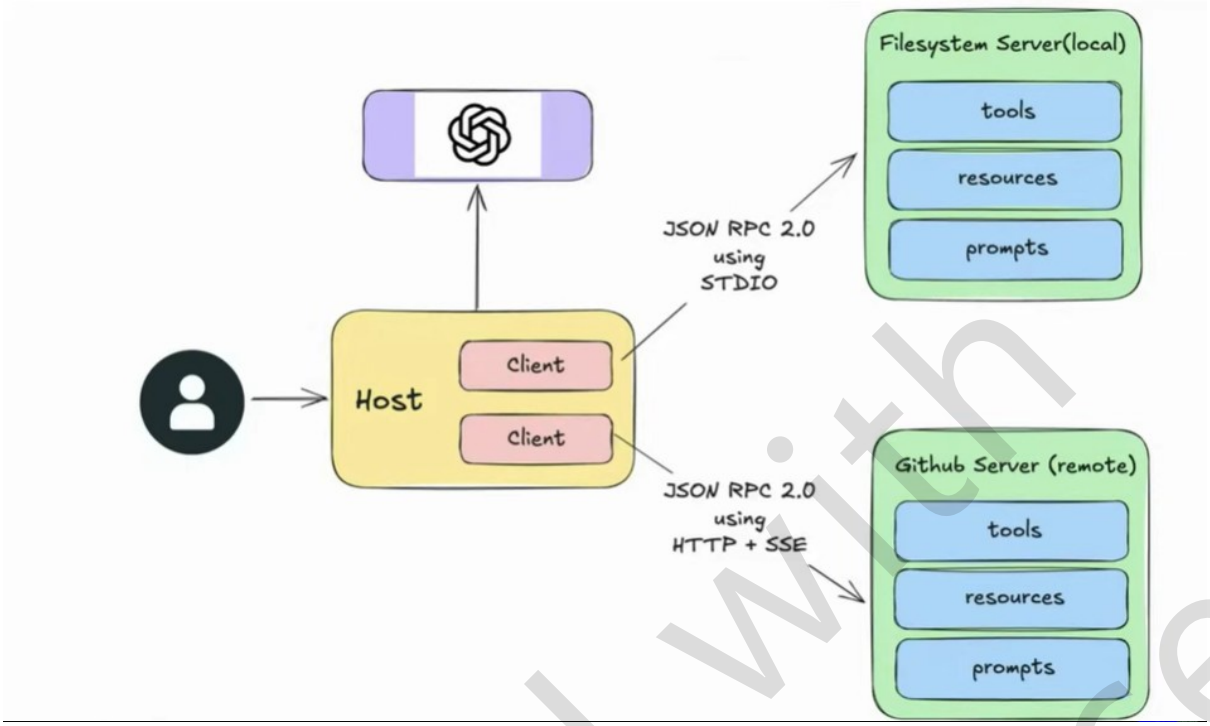
Remote Servers - HTTP + SSE

SSE stands for Server Sent Events, and it's an extension of HTTP

Using SSE the server sends multiple messages to client over a single open connection

Instead of sending one large JSON blob, the server can stream chunks of data as they are ready

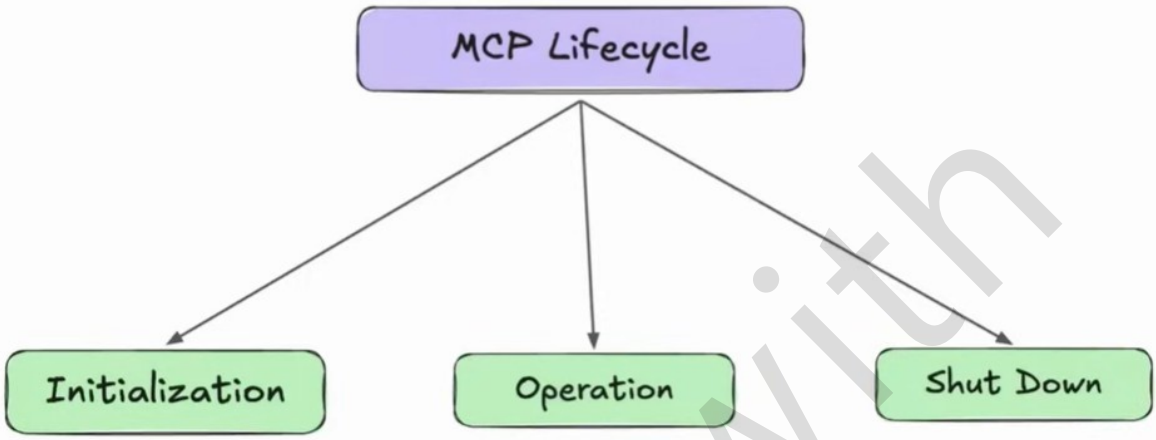
Ideal for long running tasks or incremental updates



What is MCP Lifecycle?

The **MCP Life Cycle** describes the **complete sequence of steps** that govern how a Host (client) and a Server establish, use, and end a connection during a session.

Stages of MCP Lifecycle



Initialization Phase

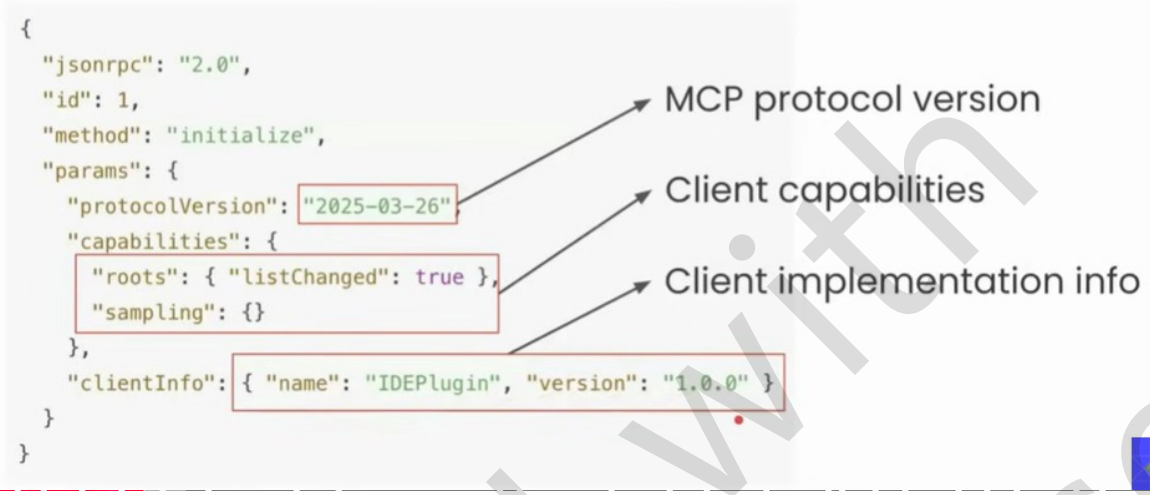
The initialization phase MUST be the first interaction between client and server.

Establish protocol version compatibility

Exchange and negotiate capabilities

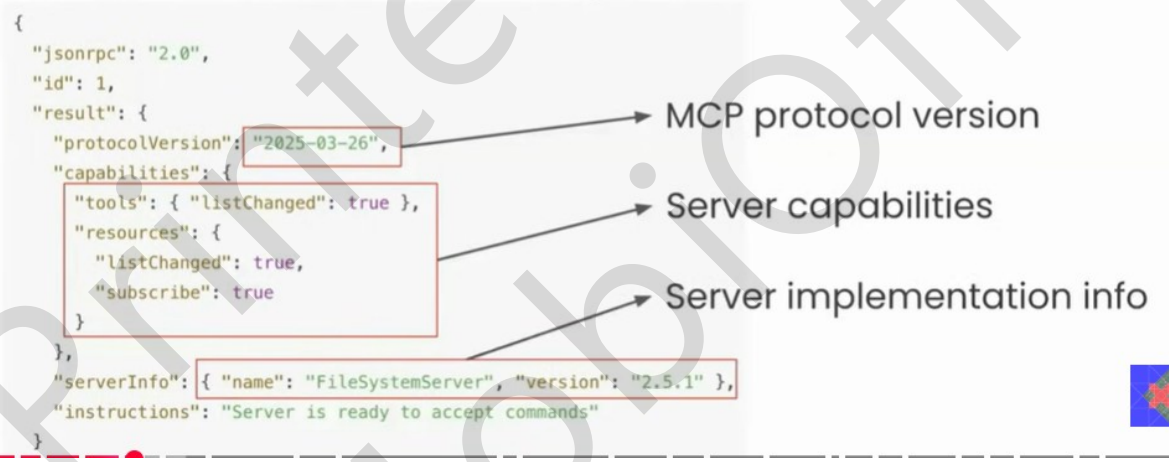
Initialization Phase - Step 1

The Client sends a *initialize* request containing:



Initialization Phase - Step 2

The Server sends its own capabilities and info:



Initialization Phase - Step 3

After successful initialization, the client MUST send an *initialized* notification to indicate it is ready to begin normal operations:

```
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

Now the Client and Server are connected

Important Rules

The client **SHOULD NOT** send requests other than **pings** before the server has responded to the *initialize* request.

The server **SHOULD NOT** send requests other than **pings** and **logging** before receiving the *initialized* notification.

Version Negotiation

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-03-26",
    "capabilities": {},
    "clientInfo": { "name": "IDEPlugin",
  }
}
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "protocolVersion": "2024-11-05",
    "capabilities": {},
    "serverInfo": { "name": "FileSystemServer"
  }
}
```

Capability Negotiation

Client and server capabilities establish which protocol features will be available during the session.

Client	`roots`	Server	`prompts`
Client	`sampling`	Server	`resources`
Client	`elicitation`	Server	`tools`
		Server	`logging`

Sub-capabilities

`listChanged`

`subscribe` :

Operation Phase

During the operation phase, the client and server exchange messages according to the negotiated capabilities.

Respect the negotiated protocol version

Only use capabilities that were successfully negotiated

Shutdown in HTTP

Client-initiated shutdown (common case):

The client (Host) **closes the HTTP connection(s)** it opened to the Server.

Server-initiated shutdown (possible):

The server may close the connection from its side

The client must be prepared to detect a dropped connection and handle it (e.g., reconnect if appropriate).

Pings

Ping is a lightweight request/response method defined in MCP.

Purpose: to check whether the other side (Host or Server) is still alive and the connection is responsive.

```
{ "jsonrpc": "2.0", "id": 42, "method": "ping" }
```

```
{ "jsonrpc": "2.0", "id": 42, "result": {} }
```

Error Handling

Error handling in MCP is how the Host (client) and Server signal that **something went wrong** with a request

MCP inherits JSON RPC's standard error object format

Causes of Error

Unsupported or mismatched protocol version.

Calling a method for a capability that wasn't negotiated.

Invalid arguments to a tool

Internal server failure while processing a request.

Timeout exceeded → client cancels request.

Malformed JSON-RPC messages.

Error Object Structure

code → integer code that categorizes the error.

message → short human-readable explanation.

```
{
  "jsonrpc": "2.0",
  "id": 5,
  "error": {
    "code": -32601,
    "message": "Method not found",
    "data": { "extra": "optional info" }
  }
}
```

Common Error Codes

Error Code	Name	Meaning	Example
-32601	Method not found	Called a method that doesn't exist or wasn't advertised	Host calls `prompts/list` but server never advertised `prompts`.
-32602	Invalid params	Request sent with wrong or missing parameters	Tool expects `{ "path": "…" }`, but client sends `{ "file": "…" }`.
-32600	Invalid Request	Malformed JSON-RPC request structure	Missing required fields like `jsonrpc` or `method`.
-32700	Parse error	JSON could not be parsed	Request body is not valid JSON.
-32000 and above	Server-defined errors	Custom errors defined by the server (implementation-specific)	Authentication failure, rate limit exceeded, quota errors, internal issues.

Timeout

Timeout is about ensuring requests don't hang forever

Purpose

Protects against unresponsive or overloaded servers.

Ensures resources (memory, CPU) aren't held indefinitely.

Gives the user feedback instead of waiting forever.

How Timeout Works

SDKs let Client sets a per-request timeout (e.g., 30s).

If the deadline passes with no result → client triggers a timeout.

Client then sends a **cancellation notification** to tell the server to stop.

Cancellation

```
{
  "jsonrpc": "2.0",
  "id": "7",
  "method": "tools/call",
  "params": {
    "name": "searchCode",
    "arguments": { "query": "MCP" },
    "_meta": { "progressToken": "tok-7" }
  }
}
```

```
{
  "jsonrpc": "2.0",
  "method": "notifications/cancelled",
  "params": { "requestId": "7", "reason": "Timeout exceeded (30s)" }
}
```

Progress Notification

Purpose: Let the client know a long-running request is still making progress.

Client includes a progressToken in the request's _meta.

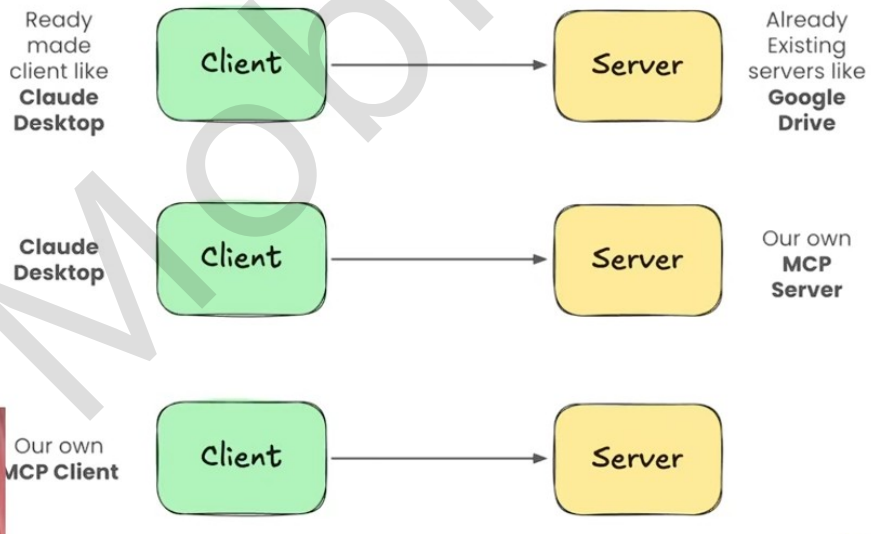
Server can then send notifications/progress updates while working.

Progress Notification

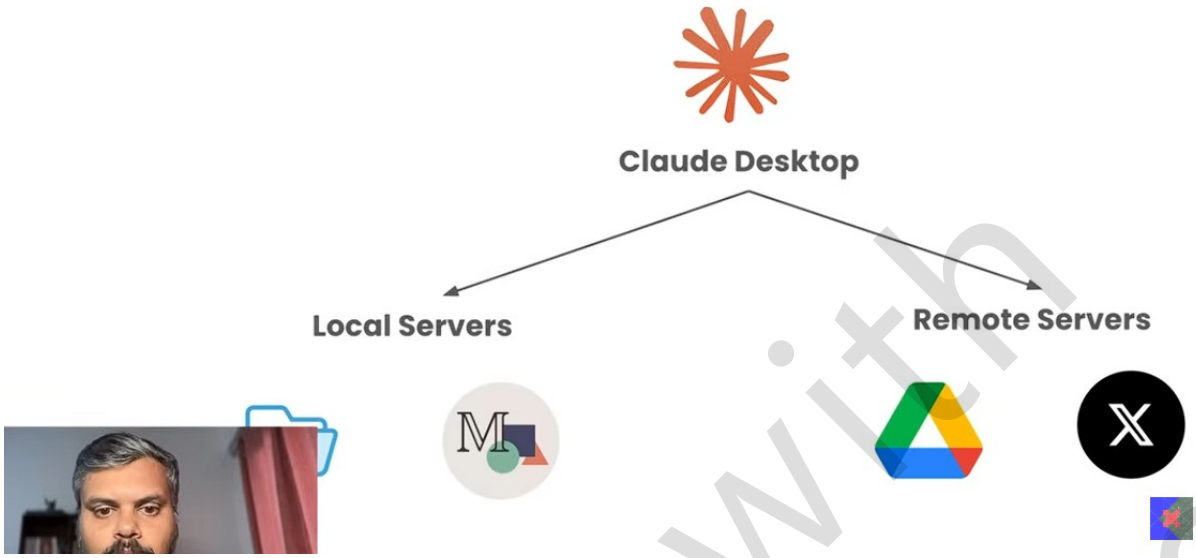
```
{
  "jsonrpc": "2.0",
  "id": 7,
  "method": "tools/call",
  "params": {
    "name": "searchCode",
    "arguments": { "query": "MCP lifecycle" },
    "_meta": { "progressToken": "tok-7" }
  }
}
```

```
{
  "jsonrpc": "2.0",
  "method": "notifications/progress",
  "params": {
    "progressToken": "tok-7",
    "progress": 60,
    "total": 100,
    "message": "Searching 600 of 1000 files"
  }
}
```

Strategy



Plan of Action



What are Connectors?

A **Connector** is a built-in feature that links Claude to MCP servers automatically, without the need for manual setup or configuration.

Most Claude Desktop users are **non-technical end-users** who just want Claude to "talk" to their apps (Notion, Google Drive, GitHub, Slack, etc.).

They don't want to run servers, edit JSON, or worry about transports.

The **Connector system** wraps an MCP server behind the scenes and handles authentication via OAuth (sign-in with Google, GitHub, etc.).

This keeps things **easy, safe, and consistent**.

Think of Connectors as the "App Store" for MCP servers — user-friendly, click-based, pre-curated.

Why not use Connectors always?

Connectors Are **Curated & Managed**

Connectors are **officially built, hosted, and maintained** by Anthropic

They come with **OAuth login flows**, managed security, rate-limits, and guaranteed stability.

If every MCP server were required to be a Connector, it would mean Anthropic has to **review, host, and secure every possible server** — which doesn't scale

MCP is an **Open Standard**

MCP is designed so *anyone* can write a server

Forcing everything through Connectors would **close the ecosystem** and make you dependent on Anthropic to approve or publish servers.